

M.Sc.(AI&ML) - I
Practical Assignment – I
Data Structures using C++

Note : (1) **Programs should have proper validations for the inputs**
(2) **Use proper variable names**
(3) **Program should contain program definition as comments and should have proper Indentation**

Classes , Objects , Functions in C++

1. Write a program to create class Student with student's rollno, name and marks of three subjects (OOCp, AI and MF) and display the details of student with total marks of all subjects along with the percentage in proper format. Output should be in descending order of percentage if student having same percentage then consider higher marks in MF should be display first.
2. Write a program to create class Num (int n1, int n2, int n3, int n4). Display total and average of n1, n2, n3 and n4.
3. Write a program to create class Time (int h, int m). Read a value as minutes from user to display new time after adding the value to minutes in Time.
4. Write a program to create class Date (int day, int month, int year). Read a value as day from user to display new date after adding the value to day in Date.
5. Write a program to create class employee with employee's id, name and basic salary. Calculate gross salary for each employee(HRA 20%, DA 30%, OA 10%). Also count total number of employee records that are entered.
6. Write a program to define a class called book. Write a program to read information about 10 books and display books details in ascending order of price in proper format.
7. Create class STUDENT having rollno, name and age as data members, also take subject with three subjects and initialize their value with minimum passing marks. Using member function, modify marks of student with specific rollno which is given by user.
8. Define a class to represent a bank account. Include the following members :

DATA MEMBERS

Name of depositor

Account Number

Type of Account

Balance amount in account

MEMBER FUNCTIONS

(1) To assign initial values

(2) To Deposit the amount

(3) To withdraw an amount after checking the

(4) To display name and balance

Write C++ program to handle 10 customers.

9. Write a program to create class 'Search' having data members (int a[], x) and define member functions as void input(), void output(), void search(int position), void add(int value) to display result. Also display how many times search method is called.
10. Write a C++ program to create a class Wallet to manage the details and transactions of digital wallets. The class should contain the following data members: Wallet ID, Owner Name, Balance, and Transaction Count.

Also display following details:

Details of the wallet having the highest balance.

Details of the wallet having the highest number of transactions.

The total number of transactions performed by all wallets.

Validate followings:

Additional Conditions:

Initial balance must be at least ₹500.

Deposit amount must be ₹100 or more.

Withdrawal is allowed only when the resulting balance remains at least ₹500.

Every successful deposit or withdrawal should increase the wallet's transaction count and the static total transaction count.

Invalid deposits and withdrawals should not be counted as transactions.

11. Write a program to create a class Product having:

Product ID

Product Name

Price

Quantity

Create records for 10 products and perform the following operations:

Display all products.

Calculate total inventory value.

Display the product having the highest price.

Display products whose quantity is less than 10.

Search a product using Product ID.

Validation:

Product ID must be unique and positive.

Price must be greater than 0.

Quantity cannot be negative.

12. Create a class Electricity Bill having:

Consumer Number

Consumer Name

Previous Meter Reading

Current Meter Reading

Calculate the electricity bill according to the following slabs:

Units	Rate
First 100	₹2/unit
101–200	₹3/unit
201–300	₹5/unit
Above 300	₹7/unit

Display:

Units consumed

Bill amount

Consumer details

Create records for 10 consumers and display the consumer having the highest bill.

Validation:

Meter readings cannot be negative.

Current reading must be greater than or equal to previous reading.

Consumer number must be positive.